

PRODUCT REQUIREMENTS DOCUMENT (PRD)

AI-Powered Automatic Block Planning System for Indian Railways

MVP Build Specification for Antigravity + Supabase

Version 1.0 | Prototype / Hackathon MVP

Purpose: Provide Antigravity with a clear, implementation-ready specification for a working prototype that integrates TMS, SMMS, TDMS, BDMS and COA data, prioritizes maintenance, detects operational conflicts, groups nearby work and recommends explainable maintenance block windows.

1. Product Vision

Create an intelligent coordination layer for railway maintenance planning. The platform should use existing departmental data rather than replace the source systems. For the MVP, CSV sample data is imported into Supabase, processed by a Data Hub, analyzed by an explainable decision engine, and presented through a dashboard where an authorized user can review and approve a recommended block plan.

2. Problem

Maintenance requirements are distributed across departmental systems. A planner may need to compare defects, severity, overdue work, historical recurrence, block requirements, nearby maintenance activities and train operations before selecting a maintenance window. The MVP should demonstrate how these datasets can be brought together to reduce manual coordination and avoid unnecessary disruption.

3. MVP Objectives

- Import and store TMS, SMMS, TDMS, BDMS and COA sample CSV data.
- Keep raw source records separate from processed records.
- Create a Data Hub that validates, cleans and standardizes the source data.
- Create a unified maintenance view using common identifiers and KM location.
- Calculate an explainable maintenance priority score using current and historical factors.
- Identify nearby maintenance activities that can potentially be combined.
- Check passenger and goods train schedules for conflicts.
- Find feasible block windows using required duration and operational availability.
- Generate a recommended block plan with an explanation.
- Allow a human user to approve, reject or modify the recommendation.

4. Scope

In Scope

- TMS, SMMS, TDMS, BDMS and COA sample data.
- Supabase PostgreSQL database.
- CSV import / seed process.
- Data cleaning and standardization.
- Maintenance priority scoring.
- Historical maintenance analysis.

- Nearby-KM grouping.
- Train conflict detection.
- Passenger vs goods operational consideration.
- Block-window recommendation.
- Explainable AI/decision notes.
- Dashboard and approval workflow.

Out of Scope for MVP

- Direct control of railway signalling, traction power or train operations.
- Automatic execution of real railway blocks.
- Unapproved access to live Indian Railways systems.
- Safety-critical autonomous decisions.
- Training a production-grade ML model without sufficient real historical data.

5. Data Sources

Source	MVP role	Important data
TMS	Track maintenance	Defect, severity, KM location, duration, status, history
SMMS	Signal & Telecom maintenance	Fault, severity, KM location, duration, status, history
TDMS	Traction / Electrical maintenance	Equipment issue, severity, KM location, duration, status, history
BDMS	Block/disconnection information	Block requirement, requested duration, status, postponement
COA	Train operations	Train type, train ID, schedule, traffic, conflict and available windows

6. Common Data Linking Strategy

Because the prototype datasets may not all contain a formal railway section field, the MVP should use common fields already present or easily derived from the sample data. The primary linking factor should be normalized KM location. Date and time should be used for operational matching. A section/corridor field may be added later if available.

- Primary geographic key: normalized location_km or from_km/to_km.
- Temporal keys: date, start_time and end_time.
- Maintenance identity: maintenance_id / asset_id / source_record_id.
- Source traceability: source_system = TMS, SMMS, TDMS or BDMS.
- COA records should cover the KM range in which maintenance occurs.

7. Supabase Database Design

Table	Purpose
tms_data	Raw TMS CSV records
smms_data	Raw SMMS CSV records
tdms_data	Raw TDMS CSV records
bdms_data	Raw BDMS CSV records
coa_data	Raw COA train and block-window records
unified_maintenance	Standardized maintenance records used by analysis
maintenance_history	Historical occurrences, duration and postponement information

Table	Purpose
priority_results	Priority score, level and explanation
maintenance_groups	Nearby compatible tasks grouped for possible combined blocks
block_recommendations	Candidate and recommended block windows
approvals	Human review decisions

8. Data Hub Requirements

The Data Hub is the integration layer between the source tables and the AI/decision engine.

- Read records from all source tables.
- Validate mandatory fields and flag invalid records.
- Normalize severity values to Critical, High, Medium and Low.
- Normalize KM values into numeric location values.
- Normalize date and time formats.
- Map different departmental field names to common fields.
- Preserve source_system and source_record_id for traceability.
- Detect duplicate records.
- Create a unified maintenance dataset.
- Expose clean data to the priority and block-planning modules.

9. Maintenance Priority Engine

The MVP should use an explainable weighted score or rule-based scoring engine rather than depending on a trained ML model. This allows judges and railway decision makers to understand every recommendation. The scoring weights should be configurable.

Priority factors

- Current severity: Critical / High / Medium / Low.
- Overdue days.
- Asset criticality.
- Defect/failure type.
- Previous occurrence count.
- Historical severity.
- Repeat postponement risk.
- Historical average and actual maintenance duration.
- Current block requirement.

Example scoring concept

Illustrative only: severity 35%, overdue status 15%, asset criticality 15%, recurrence/history 15%, postponement risk 10%, and operational/block urgency 10%. Antigravity should implement these as configurable values so the prototype can be tuned without changing application logic.

Priority output

0–39: Low **40–64:** Medium **65–84:** High **85–100:** Critical. Thresholds are prototype defaults and should be configurable.

10. Historical Intelligence

Historical records must influence priority rather than being displayed only as background information.

- Count previous occurrences of the same or related defect/asset.
- Track previous maintenance dates.
- Track actual historical completion duration.
- Track number of postponements.
- Calculate average postponement delay where available.
- Increase risk when the same critical issue repeatedly returns.
- Use historical actual duration when sizing a proposed block.

11. Nearby Maintenance Grouping

The system should identify maintenance activities located close to each other and determine whether they can potentially be performed in one coordinated block.

- Compare normalized KM locations.
- Use a configurable proximity threshold, e.g. 0.5 km for the prototype.
- Group only activities that are compatible with the same operational/block requirements.
- Show grouped maintenance IDs, departments and combined duration.
- Do not automatically combine tasks that have incompatible requirements.
- Present grouping as a recommendation for officer review.

12. COA Operational Analysis

COA is used to understand train movements and identify periods in which maintenance may cause operational conflicts.

- Include passenger trains and goods trains.
- Store train ID, train type, date, start/end time and KM coverage.
- Represent available block windows.
- Identify scheduled trains overlapping a candidate block.
- Give passenger movements higher operational consideration in the prototype when evaluating disruption.
- Consider goods trains as operational conflicts as well.
- Prefer a window with fewer or lower-impact conflicts.
- Never claim that a window is operationally safe; label it as an AI recommendation requiring approval.

13. Block Recommendation Engine

For each high/critical maintenance request, generate candidate windows and rank them.

- Determine required duration using actual historical duration where available, otherwise estimated duration.
- Add reasonable prototype buffer time if configured.
- Check whether BDMS requires a block/disconnection.
- Check COA for scheduled train conflicts.
- Check whether nearby compatible maintenance can fit in the same window.
- Rank candidate windows by priority coverage, conflict level, available duration and grouping benefit.
- Return the best candidate plus alternatives.

Example recommendation

Recommended block: 23:30–02:30. Reason: critical maintenance at nearby KM locations can be grouped, historical duration indicates that three hours is sufficient, and the candidate window has no detected scheduled-train overlap in the sample COA data.

14. Explainable AI

Every result must contain human-readable reasons. Avoid a black-box score with no explanation.

Priority explanation example

“Marked Critical because the current defect severity is Critical, the task is overdue, the asset has repeated historical occurrences, and previous postponements indicate elevated recurrence risk.”

Block explanation example

“Recommended 23:30–02:30 because nearby compatible tasks can be combined, the available window covers the required historical duration, and no scheduled passenger or goods train conflict was detected in the sample COA data.”

15. Dashboard Requirements

- Overview cards: Critical, High, Medium, Low, overdue and conflict counts.
- Maintenance table with department, location, severity, overdue days and priority score.
- Historical-risk indicators.
- Map-like KM visualization or timeline for nearby maintenance grouping.
- COA train timeline showing passenger and goods movements.
- Conflict indicators for candidate block windows.
- Recommended block card with start/end time, grouped tasks, duration and explanation.
- Approval controls: Approve, Reject, Modify.
- Filters by department, severity, date and location.

16. User Flow

1. User opens dashboard → 2. System loads Supabase data → 3. Data Hub validates and standardizes records → 4. Priority engine ranks maintenance → 5. User opens a task → 6. System finds nearby compatible tasks → 7. COA engine checks train conflicts → 8. Block engine generates candidate windows → 9. Explainable recommendation is shown → 10. User approves/rejects/modifies.

17. Antigravity Implementation Instructions

Build the MVP as a responsive web application. Keep the architecture modular so the prototype can later connect to real APIs.

- Frontend: dashboard, tables, filters, timelines and recommendation views.
- Backend/API: secure server-side logic for database access and decision calculations.
- Database: Supabase PostgreSQL.
- Data ingestion: CSV import/seed process.
- Data Hub service: validation, normalization and unified-data generation.
- Decision engine: configurable priority and block scoring.
- Explainability service: generate deterministic reason strings from the factors used.
- Approval service: store user decision and timestamp.

- Use environment variables for Supabase URL and keys; never hard-code secrets.
- Use sample/synthetic data only for the MVP unless authorized live data is available.

18. Suggested API / Service Endpoints

Endpoint	Purpose
GET /api/maintenance	List unified maintenance records with filters
GET /api/maintenance/:id	Maintenance details and history
POST /api/import	Import/validate CSV data
GET /api/priority/:id	Return priority score and explanation
GET /api/groups/:id	Return nearby compatible maintenance tasks
GET /api/coa/conflicts	Return train conflicts for a candidate window
POST /api/block/recommend	Generate candidate and recommended windows
POST /api/approvals	Store approve/reject/modify decision

19. Key Acceptance Criteria

- Five source datasets can be imported and viewed.
- Raw source data remains traceable to its original system.
- Data Hub produces standardized unified maintenance records.
- Current and historical factors change the priority score.
- Critical, High, Medium and Low outputs are visible.
- Nearby KM maintenance activities can be grouped.
- Passenger and goods train conflicts are detected.
- Candidate block windows are generated.
- A recommended window is selected using configurable logic.
- Historical actual duration affects block sizing when available.
- Postponement/recurrence risk affects priority.
- Every recommendation contains an explanation.
- Officer approval/rejection/modification is stored.
- No AI recommendation directly controls real railway operations.

20. Demo Scenario for Judges

Use a sample scenario where TMS, SMMS and TDMS contain maintenance tasks within nearby KM locations. BDMS contains the corresponding block requirement. COA contains passenger and goods train movements followed by an available overnight window. The system should detect that the maintenance tasks are geographically compatible, calculate their priorities using current and historical data, detect conflicts during busy periods, and recommend the available window where the combined work can be completed. The dashboard should show exactly why the recommendation was produced.

21. Security and Safety

- Use Supabase authentication/authorization for the prototype if user accounts are implemented.
- Keep database credentials and API keys in environment variables.

- Use least-privilege database access.
- Treat all AI output as decision support.
- Display a clear “Requires authorized officer approval” status on recommendations.
- Do not connect the MVP to live railway control systems.

22. Future Enhancements

- Live authorized integrations with railway systems.
- Machine-learning model trained on validated historical data.
- Advanced optimization for multiple simultaneous blocks.
- GIS railway map and corridor visualization.
- Real-time train movement data.
- Weather and disruption risk.
- Feedback loop from officer decisions to improve recommendations.
- Role-based workflows across departments.

23. Final MVP Definition

The completed MVP is a web-based AI-assisted railway maintenance planning platform that integrates TMS, SMMS, TDMS, BDMS and COA sample data through a Data Hub and Supabase database. It uses current and historical maintenance information to prioritize work, groups nearby compatible activities, checks passenger and goods train conflicts, recommends suitable block windows, explains the reasoning behind each recommendation, and records final human approval. The prototype demonstrates intelligent coordination without replacing or autonomously controlling existing railway systems.